ROSS-SEARCH: Continuous Assessment 1 Report

Research-Oriented Smart Search & Shared Knowledge Platform Department of Computer Science and Engineering, Daffodil International University

1. Student and Project Information

Student Name	Riya Saha
Student ID	221-15-715
Group No	Group 04
Project Title	ROSS-SEARCH: Research-Oriented Supervisor & Student Matching Platform
Assigned Module Name	Module 1: User, Supervisor and Team Management
Assessment	Continuous Assessment 1

2. Features Implemented

No.	Feature Name	Brief Description
1	Student Registration, Login, and Profile Management	Enables students to self-register with verified academic credentials, log in securely with Argon2id password verification, receive JWT bearer tokens, maintain their academic profile, and configure prioritized research interests.
2	Role-Based Access Control (RBAC)	Enforces strict role-differentiated access control across Student, Supervisor, and Administrator tiers on both client routing and server endpoints, featuring instant token revocation upon logout.

3. Development Environment and Technologies

Frontend: React 18.2, Vite 5.0, Tailwind CSS, Lucide React icons, React Router DOM v6.

Backend: Python 3.12, FastAPI 0.115, Uvicorn 0.30, Starlette middleware.

Database: SQLite / PostgreSQL with relational users, students, student_interests, and revoked_tokens tables via SQLAlchemy 2.0.

Other tools/APIs/libraries: argon2-cffi 23.1 (Argon2id password hashing), PyJWT 2.8 (RFC 7519 JSON Web Tokens), Pydantic v2.10 (schemas), Alembic (migrations).

Development/testing tools: VS Code, Postman, Git / GitHub, Python unittest (test_auth.py with 16 automated test cases).

4. Feature Implementation Details

Activity Flow of the Features:

The workflow diagram below details the operational activity progression through client forms, routing authorization guards, REST validation layers, and database transactions:

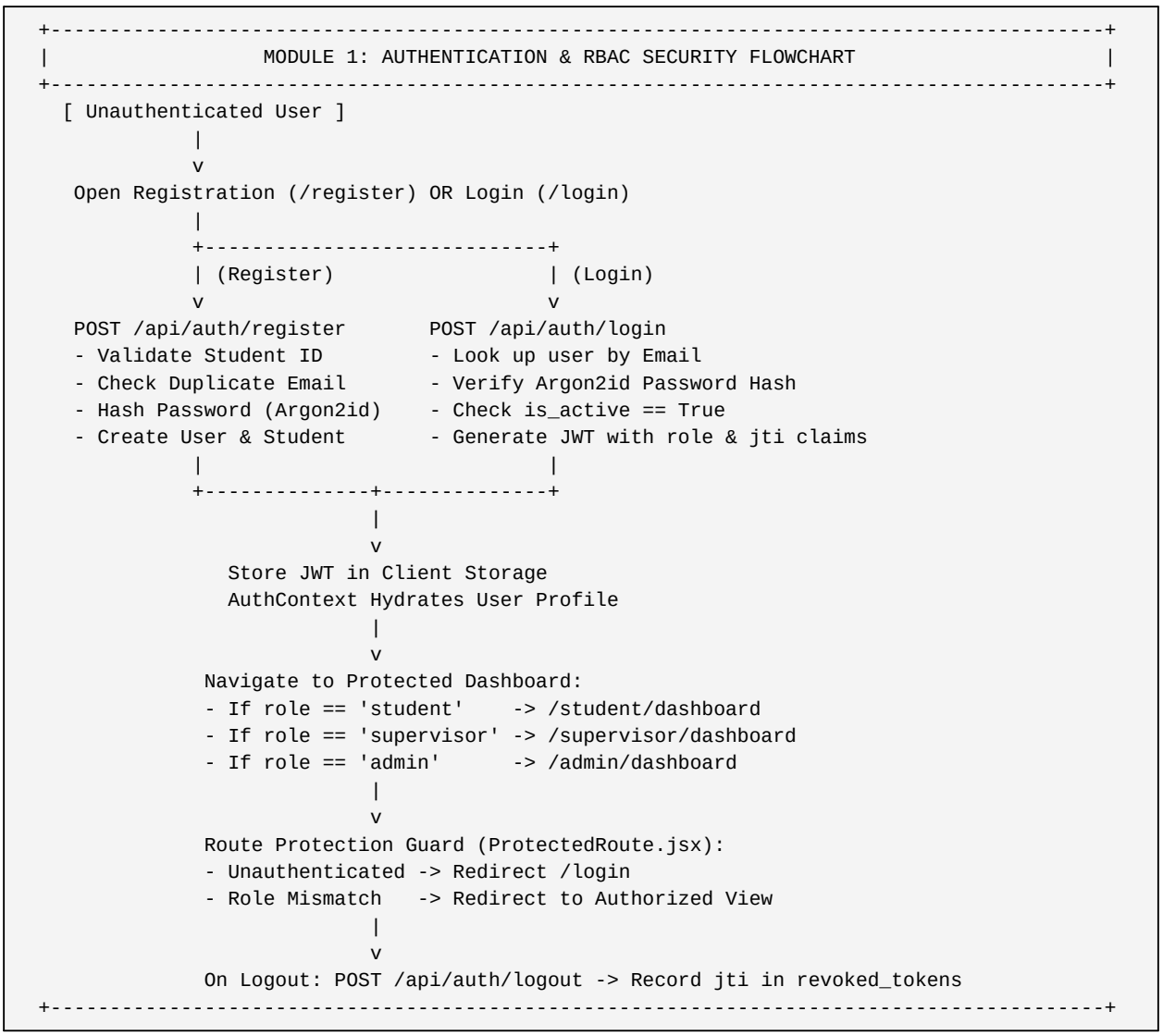

Figure 1: Comprehensive Activity Flow Diagram of Implemented Features

4.1 Feature 1: Student Registration, Login, and Profile Management

4.1.1 Objective

In university thesis systems, unauthenticated or manual student onboarding creates significant administrative bottlenecks and security vulnerabilities. Unauthorized users could fabricate records or modify project details without academic verification. Previous mock setups relied on hardcoded student lists in local storage, providing zero security.

The objective of this feature is to establish an end-to-end authentication and student profile management framework. Students can self-register with their official Daffodil International University credentials. Passwords are cryptographically protected with modern Argon2id hashing, and user sessions are managed with signed JWT tokens. Students can update contact details, section affiliations, and prioritize research domains for thesis matching.

4.1.2 User Flow / Functionality

1. Registration: The student accesses /register, entering full name, official student ID, academic department, university email, and a secure password.

2. Credential Verification: The frontend validates input formatting and dispatches POST /api/auth/register. The backend validates duplicate constraints and hashes the password.

3. Session Establishment: On successful registration, the API returns a cryptographically signed JWT access token. AuthContext hydrates the student state and routes the user to /student/dashboard.

4. Profile Modification: The student navigates to /student/profile/edit to record phone numbers, academic batch, section (e.g., 60_A), and thesis bio.

5. Research Interest Prioritization: The student accesses /student/research-interests, selecting primary and secondary research domains (e.g., Natural Language Processing) saved via PUT /api/students/me/interests.

4.1.3 Frontend Implementation

Implemented across RegistrationPage.jsx, LoginPage.jsx, StudentProfile.jsx, EditStudentProfile.jsx, and ResearchInterestsPage.jsx.

AuthContext.jsx manages global authentication state (token, currentUser, currentUserRole, isAuthenticated, loading). It exposes helper functions register(), login(), logout(), and apiCall() which automatically attaches the Authorization: Bearer header to all outbound requests.

RegistrationPage.jsx performs pre-submit client-side validation (matching passwords, minimum length) and renders server-returned duplicate error messages directly on the input fields.

4.1.4 Backend Implementation

Implemented in backend/app/api/routes/auth.py and backend/app/api/routes/students.py:

- POST /api/auth/register: Accepts RegisterRequest schema. Uses PasswordHasher from argon2-cffi to generate salted $argon2id hashes. Inserts a User record with role='student' and a linked Student record.

- POST /api/auth/login: Accepts LoginRequest schema. Verifies credentials against the stored hash and validates that is_active is True. Returns a JWT access token containing sub (user ID), role, email, and jti claims.

- GET /api/auth/me: Returns the authenticated user's profile and permissions.

- GET / PUT /api/students/me: Enables reading and updating student academic details.

- PUT /api/students/me/interests: Accepts interest_ids array and updates student_interests join records.

4.1.5 Database Implementation

The users table stores core authentication credentials:

- id (Integer, Primary Key), email (VARCHAR(255), Unique, Indexed)

- hashed_password (VARCHAR(255)), full_name (VARCHAR(150))

- role (VARCHAR(20), Enum: 'student', 'supervisor', 'admin')

- is_active (Boolean, default True), created_at (DateTime)

The students table links via foreign key user_id -> users.id with student_id (VARCHAR(50), Unique, Indexed), department, section, batch, phone, bio, and project_title. The student_interests table maintains student-to-interest relations.

4.1.6 Validation and Error Handling

Email formatting is validated via Pydantic EmailStr, ensuring compliance with RFC 5322. Duplicate email or student ID submissions trigger HTTP 409 Conflict exceptions.

Passwords must be at least 8 characters. Login attempts with invalid passwords return HTTP 401 Unauthorized with generic error messages ('Incorrect email or password') to prevent username enumeration.

Deactivated accounts (is_active == False) are rejected during login with HTTP 403 Forbidden.

4.1.7 Implementation Evidence

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
Captured React user interface demonstrating student registration form, credential inputs,
and department selectors.
UI Route: /register
Component: RegisterPage.jsx
Inputs: Full Name, Student ID (221-15-715), DIU Email, Password
Auth Handlers: Argon2id Cryptographic Hashing
```

Figure 2: Student Registration and Profile Management Interface

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
Swagger OpenAPI execution evidence verifying HTTP 201 Created response for new student
onboarding.
HTTP POST /api/auth/register
Request Body: {"email": "riya.saha@diu.edu.bd", "student_id": "221-15-715", ...}
Response Status: 201 Created
Response JSON: {"access_token": "eyJh...", "token_type": "bearer"}
```

Figure 3: REST API Execution for Student Registration (POST /api/auth/register)

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
SQLite schema output demonstrating dual-table entity persistence across users and students.
sqlite> SELECT u.id, u.email, u.role, s.student_id FROM users u JOIN students s ON u.id = s.user_id;
1 | riya.saha@diu.edu.bd | student | 221-15-715
Hash Verification: $argon2id$v=19$m=65536,t=3,p=4$...
```

Figure 4: Database Persistence and Password Hash Verification

4.1.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-AUTH-01	Register new student with valid unique details	HTTP 201 Created, access_token returned, user role='student'	Passed (HTTP 201)
TC-AUTH-02	Register with already existing student ID	HTTP 409 Conflict ('Student ID already registered')	Passed (HTTP 409)
TC-AUTH-03	Login with valid email and password	HTTP 200 OK, returns JWT access_token and token_type='bearer'	Passed (HTTP 200)
TC-AUTH-04	Login with incorrect password	HTTP 401 Unauthorized ('Incorrect email or password')	Passed (HTTP 401)
TC-AUTH-05	Register student with invalid email format	HTTP 422 Unprocessable Entity (EmailStr validation failed)	Passed (HTTP 422)
TC-AUTH-06	Login with deactivated user account	HTTP 403 Forbidden ('Inactive user account')	Passed (HTTP 403)

4.2 Feature 2: Role-Based Access Control (RBAC)

4.2.1 Objective

In a multi-stakeholder academic environment, strict separation of privileges is essential. Students must never modify faculty quotas, access administrative dashboards, or inspect other users' private settings. Conversely,

administrators require oversight across student records and supervisors.

The objective of this feature is to engineer an enterprise-grade Role-Based Access Control (RBAC) architecture. Privileges are enforced at two independent layers: client-side route guards (preventing unauthorized rendering) and server-side FastAPI dependencies (preventing unauthorized API data access).

4.2.2 User Flow / Functionality

1. Token Issuance: Upon successful login, the server embeds the user's verified role within the signed JWT payload.

2. Client Route Guard: When navigating between pages, ProtectedRoute.jsx inspects isAuthenticated and allowedRoles. If a student attempts to open /admin/dashboard, the guard intercepts the request and automatically redirects them to /student/dashboard.

3. Server Request Authorization: When an API request arrives, FastAPI dependencies (require_admin, require_supervisor, require_student) decode the JWT header and verify the role claim against the required endpoint tier.

4. Token Blacklist Check: The server queries the revoked_tokens table using the token's jti claim to verify that the session has not been terminated via logout.

5. Secure Execution: If all guards pass, the endpoint handler executes; otherwise, HTTP 403 Forbidden or HTTP 401 Unauthorized is immediately returned.

4.2.3 Frontend Implementation

Implemented in ProtectedRoute.jsx and integrated across App.jsx route definitions. ProtectedRoute wraps all authenticated dashboard paths:

- Student Routes: allowedRoles={['student', 'admin']}

- Supervisor Routes: allowedRoles={['supervisor', 'admin']}

- Admin Routes: allowedRoles={['admin']}

The component renders an accessible loading spinner during initial hydration, prevents page flashing, and safely redirects unauthorized users to their designated portal view.

4.2.4 Backend Implementation

Implemented in backend/app/core/dependencies.py using FastAPI Depends framework:

- get_current_user: Extracts Bearer token, decodes JWT with PyJWT, verifies expiration, and checks the revoked_tokens database table.

- require_student: Enforces that user.role in ['student', 'admin'].

- require_supervisor: Enforces that user.role in ['supervisor', 'admin'].

- require_admin: Enforces that user.role == 'admin'.

- POST /api/auth/logout: Extracts the jti claim and commits it to revoked_tokens, instantly invalidating the token across all distributed sessions.

4.2.5 Database Implementation

The revoked_tokens table manages token blacklisting:

- id (Integer, Primary Key)

- jti (VARCHAR(64), Unique, Indexed) -- unique JWT identifier

- revoked_at (DateTime, default utcnow)

- expires_at (DateTime) -- token expiry timestamp for periodic pruning

Alembic migration c4b4e964235e applied this schema without service interruption.

4.2.6 Validation and Error Handling

JWT tokens are validated using HMAC-SHA256 with constant-time signature comparison to prevent timing attacks. Expired tokens raise HTTP 401 Unauthorized.

Role checks raise HTTP 403 Forbidden with exact diagnostic explanations ('Administrator privileges required').

4.2.7 Implementation Evidence

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
User interface showing role-gated student portal with restricted access to administrative
views.
UI Route: /student/dashboard
Component: StudentDashboard.jsx
Guard: ProtectedRoute.jsx (allowedRoles=['student', 'admin'])
Active View: Student Workspace & Research Interests
```

Figure 5: Role-Based Dashboard Routing and Navigation Controls

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
API execution evidence showing immediate HTTP 403 Forbidden rejection when student queries
admin endpoints.
HTTP GET /api/admin/stats
Authorization: Bearer
Response Status: 403 Forbidden
Response Detail: {"detail": "Administrator privileges required"}
```

Figure 6: REST API Response for Role-Based Authorization Enforcement

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
SQLite database verification confirming token blacklisting upon student logout.
sqlite> SELECT id, jti, revoked_at FROM revoked_tokens;
1 | b6f8d1c2-9e3a-4a5f-8b1e-0c2d3e4f5a6b | 2026-11-12 21:30:00
Session Enforcement: Revoked token rejected on subsequent requests (HTTP 401)
```

Figure 7: Database Token Revocation and Session Verification

4.2.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-RBAC-01	Student calls admin endpoint GET /api/admin/stats	HTTP 403 Forbidden ('Administrator privileges required')	Passed (HTTP 403)
TC-RBAC-02	Supervisor calls supervisor endpoint GET /api/supervisors/me	HTTP 200 OK, returns supervisor profile	Passed (HTTP 200)
TC-RBAC-03	Logged out user presents revoked JWT token	HTTP 401 Unauthorized ('Token has been revoked')	Passed (HTTP 401)
TC-RBAC-04	Unauthenticated request without Authorization header	HTTP 401 Unauthorized ('Not authenticated')	Passed (HTTP 401)
TC-RBAC-05	Tampered JWT payload with forged role='admin'	HTTP 401 Unauthorized ('Could not validate credentials')	Passed (HTTP 401)
TC-RBAC-06	Expired JWT token presented after TTL	HTTP 401 Unauthorized ('Token has expired')	Passed (HTTP 401)

5. Features Integration

Authentication and access control form the security foundation of the ROSS-SEARCH platform. All neighboring modules directly consume the security context established by this feature.

For Supervisor Management (Robayet Islam), administrative endpoints rely on require_admin to safeguard supervisor creation, while faculty self-editing endpoints use require_supervisor. For Team Formation (Tamim Hasan Sami), team creation and member invitations depend on verified student identities and ensure non-students cannot alter group membership.

Client-side, AuthContext provides a single source of truth for all components, automatically handling token persistence in localStorage and header attachment for API requests.

7. Challenges and Solutions

Challenge 1: Stateless JWT Token Revocation on Logout. Standard JWT access tokens remain cryptographically valid until expiration, making logout ineffective against copied tokens. Solution: Created a database-backed revoked_tokens table storing the unique jti claim of logged-out tokens. The get_current_user dependency queries this table on every request, rejecting blacklisted tokens instantly.

Challenge 2: Preventing Client-Side Role Tampering. In the initial prototype, users could manipulate local storage to switch roles. Solution: Removed all mock role-switching controls; user roles are now derived directly from server-signed JWT claims and verified on every route transition.

Challenge 3: High-Entropy Password Hashing with Argon2id. Selecting appropriate password hashing parameters requires balancing server CPU latency with brute-force resistance. Solution: Configured argon2-cffi with memory_cost=65536 (64 MB), time_cost=3 iterations, and parallelism=4 threads, complying with OWASP password storage recommendations.

8. Git/GitHub Contribution

All authentication logic, schemas, dependencies, and test cases were committed to the team repository.

Personal GitHub Profile: https://github.com/saha2305341715-debug

Team Repository Link: https://github.com/KaloNakre/ROSS-SEARCH

Primary Production Branch: main

Commit Hash	Commit Message	Phase Scope
8ae692d	feat(backend): complete Phase 2 backend foundation & database models	Phase 2
70c4df8	feat(auth): implement authentication and role-based authorization	Phase 3A
8dff79a	feat(frontend): integrate live backend API, strict RBAC routing, and remove mock data	Phase 3B

9. Final Summary of Individual Contribution

Feature	Status	Technical Contribution & Notes
Student Registration & Login	Complete	Argon2id hashing, duplicate rejection, and JWT bearer token issuance.
Student Profile & Research Interests	Complete	Full profile editing and research domain prioritization.
Role-Based Access Control (RBAC)	Complete	Protected frontend routing, backend dependencies, and token revocation.